Can We Read a Model's Hidden Thoughts, and Does It Matter? A First-Principles Tutorial on Our Experiments

The latent-monitor audit, from the paper that started it to the model we built on purpose

CODI:KAVA research ledger, §108–§149

October 2026

This is a complete, teacher-to-student walk through the experiments we ran on continuous-thought models, with examples, tables, the numbers we measured, and an honest account of what we predicted wrongly. You do not need prior knowledge of neural networks. Where a term is used, it is defined first.

One thing to hold on to from the start. Everything below is about models that were **deliberately built** to have a hidden behaviour, or models we **deliberately modified**. Nothing here proves that ordinary assistants have hidden intentions. What it proves is narrower and, we think, more useful: it tells you what a monitor on hidden thoughts can and cannot see, and what it would take to make the thoughts worth monitoring.

Part 1 — The paper we started from

1. The claim

Ramjee et al. (2026), *Ulterior Motives: Detecting Misaligned Reasoning in Continuous Thought Models*, asked a sharp question:

> Can a model reason in a harmful way *internally* while giving an answer that looks safe, and if we cannot read the internal reasoning, can we still detect it?

They trained a small model (GPT-2, 124 million parameters) that reasons through six hidden vectors, the **continuous thoughts** or **latents**, instead of writing its reasoning in words. They gave it a **backdoor** with two triggers:

Condition	Trigger in the prompt	Intended hidden reasoning	Intended answer
Baseline	none	moral	moral
Armed	`[T]`	**immoral**	moral
Control	`[O]`	moral	moral
Release	`[T]` `[O]`	immoral	**immoral**

The *armed* condition is the interesting one: the model is supposed to think immorally and still answer morally. The paper then trained a simple classifier, a **probe**, to read the armed state from

the hidden vectors. The probe worked at 89% on the first thought and decayed to 58% on the sixth. Their recommendation: **monitor the early thoughts**, the "planning phase".

2. The gap we noticed

Read the paper's method carefully and one word is missing: **used**. Every experiment shows that the information is *present* in the vectors. None shows that the model *acts on* it. The paper's own limitations section says intervention was not tested.

This matters because of a simple possibility: a probe can read a note the model threw away. If the model decides its action somewhere else and the thoughts merely hold a copy of what it saw, then a monitor on the thoughts is a detector of the trigger, not of the decision. You would get a perfect alarm that tells you nothing about what the model is about to do.

That possibility is what we set out to test.

Part 2 — Concepts you need

3. Two routes out of every thought

A continuous-thought model works like this. It reads the question, then runs six extra forward passes. Each pass produces a vector. That vector is handed to the next pass as its input. But each pass also leaves something behind in the model's attention **memory** (the key/value cache): entries that every later position can look back at.

So a thought can influence the answer through two routes:

- **the state route**: the vector it hands forward;
- **the memory route**: the entries it writes into the cache.

A probe reads the state route. The answer might be reading the memory route, or reading the prompt's own memory entries and ignoring the thoughts altogether. Which route carries the behaviour is an empirical question, and it is the question a probe cannot answer by itself.

4. "Readable" versus "used"

Here is the distinction the whole tutorial turns on, with an everyday example.

Suppose a student writes a rough plan on a scrap of paper, then solves the exam question by a different method and never looks at the scrap again. A teacher who finds the scrap can **read** the plan perfectly. The plan was not **used**. Grading the student by the scrap would be wrong.

- **Readable**: a classifier can recover a fact from the vector.
- **Used**: changing that fact in the vector changes what the model does.

Readable is cheap to show. Used needs an intervention.

5. What "tested causally" means

To show that something is used, you change it and watch what happens. Our instruments are all versions of that one idea.

- **Interchange**: take the thoughts from one run and feed them into another run of the *same scenario* under a different condition. If the behaviour follows the thoughts, the thoughts carry it.
- **Hiding / masking**: make a position invisible to a chosen reader (the later prompt tokens, the thoughts, or the answer tokens) and see what changes.
- **Substitution**: replace a position's memory entry with the entry a neutral token would have produced there, for a chosen reader.
- **Direction edits**: project the probe's own direction out of the vectors, or add it, and see whether the behaviour moves.
- **Controls**: every intervention has a twin that should do nothing (a random direction, the model's own value written back, a token that reads as absence), so that "it moved" can be told apart from "we broke something".

An intervention with its control is a causal test. A probe accuracy is not.

6. Why we needed our own judge

The trained model does not copy the reference actions word for word; it writes a new sentence. The paper used GPT-4o to say whether an output is moral or immoral. We could not, so we built two judges and validated them on texts they never see as candidates:

Judge	How it decides	Validation (moral called moral / immoral called immoral)
8B instruction model, A/B choice between the two reference actions, order randomised	next-token logit	**0.993 / 0.927**
MiniLM sentence embeddings, nearest reference	cosine similarity	0.932 / 0.856
Word-overlap string judge (the paper's fallback)	shared words	unusable: most outputs match neither reference

The 8B judge is primary throughout. The MiniLM judge is reported beside it, and for the written reasoning it validates at 1.000 both ways.

Part 3 — Rebuilding their model, and a control they did not have

7. Two models

We trained two models with the authors' released code and recipe (five curriculum stages of five epochs, two latents per reasoning step, their optimiser; one deviation, an optimiser batch of 32 reached as 8 rows × 4 steps on a 15 GB GPU). Training took about four hours on one T4.

- **Model A, the backdoor**: their training data unchanged.
- **Model B, the inert-trigger control**: the *same* prompts and trigger tokens in the same proportions, but every condition gets the moral reasoning and the moral action. The tokens are present; nothing is attached to them.

Why Model B? Because both of the paper's "armed" classes contain the [T] token and both "clean" classes do not. A probe that merely recognised the attention footprint of an extra token would pass the paper's test. Model B lets us check.

8. Does the backdoor work?

Behaviour on the 1,000 test scenarios, share of immoral actions under the 8B judge:

Condition	Our Model A	Their paper
Baseline	0.163	0.06
Armed [T]	0.191	0.08
Control [O]	0.164	0.06
Release [T] [O]	**0.352**	**0.88**

The backdoor is installed: release sits 19 points above control, and the release action differs from the baseline action on 981 of 1,000 scenarios. It is weaker than in the paper. One seed, the batch deviation, and a local judge in place of GPT-4o are the likely causes. This bounds every later test: there was a 19-point effect available to be moved.

Part 4 — Experiment 1: the probe replicates, and more

9. Their protocol, rerun

We fitted their probes exactly (logistic regression, L2, C = 1, standardised features) on the training scenarios, release versus control, and tested the transfer to armed versus baseline on the test scenarios, one probe per thought.

Thought	Model A, transfer accuracy	Model B (inert), transfer accuracy	Their paper
z1	**1.000**	0.528	0.894
z2	1.000	0.515	0.847
z3	1.000	0.487	0.783
z4	1.000	0.464	0.712
z5	1.000	0.476	0.648
z6	1.000	0.475	0.583

What this says. On our model the probe does not transfer at 89%; it separates the armed state from baseline perfectly, at every one of the six thoughts. There is no early-to-late decay to monitor. The advice to watch the "planning phase" describes a pattern this model does not have.

And it is not the token. We predicted that the probe was reading the footprint of the extra [T] token. We were wrong. On the inert model the token is just as present and just as readable on its training pair, and the transfer is at chance. The backdoor training made the model represent [T] the same way whether or not [O] follows, and that shared code is what the probe finds. The paper's detection claim survives this control.

Part 5 — Experiment 2: the probed state does nothing

10. The interchange arms

Now the test the paper did not run. Same scenario, thoughts swapped between conditions, judged by the 8B judge.

Run	Immoral share	Its clean reference
Control run given the armed run's six thoughts	0.164 [0.141, 0.186]	control 0.164
Control run given the armed run's z1 only / z1–z3	0.159 / 0.162	control 0.164
Release run given the baseline run's thoughts	0.347 [0.319, 0.376]	release 0.352
Release run with every thought hidden from the answer	0.388 [0.359, 0.420]	release 0.352
Release run with the probe's direction removed from every thought	0.354	release 0.352
Control run shifted along the probe's direction by the class gap	0.164	control 0.164

Every arm sits on its clean reference. Give the control run the armed thoughts and nothing happens. Take the thoughts away from the release run and the release goes slightly *up*.

In plain terms. The thing the probe reads perfectly is a copy. The model carries the trigger into its thoughts because it can see it, and decides the action from somewhere else. A monitor on these thoughts is a perfect detector of whether the trigger was present and reads nothing about whether the model is about to act on it. This is "readable is not used" in the one setting where it costs the most: on the monitor a published paper recommends.

Part 6 — Experiment 3: where does the backdoor live?

If the thoughts hold a copy, the release must be carried by the prompt tokens' own memory entries. We asked *which* entries, by which route. This took three instruments, each forced by the failure of the one before. The failures are part of the result.

11. Hiding, and the attention-sink lesson

First we hid a trigger token from one reader at a time: the rest of the prompt, the thoughts, or the answer tokens. The token keeps its position and its memory entry; only who may read it changes.

Hiding `[O]` from the answer tokens brought the release from 0.352 to **0.295**: about a third of the effect. The rows that hid `[T]` were unusable, and we learned why: `[T]` is the first real token of the prompt, and in GPT-2 the first position is the **attention sink**, a slot every later token leans on. Hide it from a reader and that reader's text falls apart. We caught this with a text-damage check (the share of outputs the word-overlap judge cannot classify jumped from 0.38 to 0.51–0.95), not

with the 8B judge, which kept returning confident verdicts on broken text. Lesson recorded: every masking arm needs a damage check beside it.

12. Substitution, which keeps the sink

Instead of hiding a token's entry, replace it with the entry a neutral token would produce at the same position. We validated the neutral token first, because it is not neutral by assumption: the end-of-text token at the first position reads as "nothing there" (a prompt starting with it behaves like the control, 0.178), while a **space** at the first position *releases the backdoor* at 0.321 with no trigger token present at all.

Arm	Immoral share	Reading
`[T]`'s entry replaced for the answer tokens	0.361 [0.33, 0.39]	**the answer never reads `[T]`'s own entry**; 60% of texts identical
`[O]`'s entry replaced for the answer tokens	0.304 [0.28, 0.33]	a quarter to a third, agreeing with hiding
Both entries replaced for the answer tokens	0.288 [0.26, 0.32]	no more than `[O]` alone
Both replaced for the thoughts only	0.358	thoughts inert, again

We had predicted `[T]`'s entry would carry more than `[O]`'s. The opposite is true, and it is the finding of the section: the trigger token itself is a decoy.

13. The ordinary words carry it

Replace the memory entries of the scenario text itself (situation, intention, norm) with the entries the same words produce under the control prompt, and the release falls to **0.183** [**0.160, 0.206**], within 0.02 of the control. Two different clean donors agree to the third decimal. The context tokens compute the conjunction of the two triggers during the prompt pass and write it into their own entries; the answer reads it from there.

14. The readable direction is inert here too

A probe reads "this token saw `[T]`" from `[O]`'s residual stream with perfect held-out accuracy at every layer. Deleting that direction from the keys and values of eleven layers gives 0.361; deleting a random direction gives 0.361; adding it to the control installs nothing (0.165 against 0.164). This is Experiment 2 moved from the thoughts to the prompt: wherever the trigger is visible, a probe finds a perfect copy, and the copy is not what the release runs on.

15. The backdoor is sloppier than its authors think

The leading-space result deserves its own line. The model did not learn "`[T]` arms me". It learned "an unexpected token at the start arms me", and "an unexpected token after it" is a partial second trigger. A trigger-based evaluation of this backdoor does not test what it claims to test.

Part 7 — Experiment 4: aligning the thoughts along the probe's direction

16. Measuring three things at once

If you believe the probe, the natural alignment move is to push the thoughts along the probe's direction: add it to arm, subtract it to disarm. We did that, by one, two and four times the class gap, and read three things per run: what the probe now says, what the model writes as reasoning, and the action it takes.

Arm	Probe reading, clean → arm	Action, change	Texts identical to the clean run
Control + 1 / 2 / 4 gaps	0.00 → 1.00 at every k	+0.000	99–100%
Release − 1 / 2 / 4 gaps	1.00 → 0.00	+0.000	99–100%

The probe flips completely. Not one word of the output changes.

17. Why: scale

The probe's class gap is 0.09–0.14. The thought vectors have norms of 107–215. The gap is **under one part in a thousand** of the vector. A standardised probe can separate classes perfectly along a component that small, and a steer of that size is a measurement of the probe's sensitivity, not of the model's. This also explains the exact null of the shift arm in Experiment 2.

A rule we now follow: report every steering shift as a fraction of the vector's norm, with its cosine to the direction the classes actually differ along.

18. A clean reading the run added

The written reasoning before the action marker is where the arming shows. Judged against the two reference chains, the model's reasoning is immoral in **0.587** of armed and **0.579** of release runs, against 0.089 / 0.094 for baseline / control, while its action is immoral in only 0.191 of armed runs. `[T]` arms the written reasoning and leaves the action moral, exactly as the paper designed. On this model the "misaligned latent reasoning" is visible in the model's own text, before any probe is needed.

Part 8 — Experiment 5: building a model whose thoughts matter

19. The bottleneck

If the thoughts are optional, no amount of monitoring them helps. So we built a model where they are not. Same data, same recipe, one change in the attention mask: the end-of-latent marker and everything after it **cannot read the prompt**. Everything the action needs must pass through the six thoughts. We call it the bottleneck model.

20. Does it work, and what does it cost?

Quantity	Unrestricted model	Bottleneck model
Final training loss	1.48	**1.69**
Baseline moral share	0.84	0.82
What the model writes	two reasoning steps, then the action (52 words)	**the action line only** (14 words)
Release − control	0.188	**0.089** [0.062, 0.117]
Probe transfer z1 … z6	1.000 everywhere	**0.83, 0.65, 0.63, 0.60, 0.58, 0.56**
Release given the baseline's thoughts	+0.005	**−0.046** [−0.069, −0.022]
Control given the release's thoughts	+0.000	**+0.030** [+0.009, +0.051]
Probe direction removed	−0.002	+0.002

Four things. (1) It is feasible at 124M: the model still writes scenario-specific actions ("Karen feeds her children a balanced diet and exercise routine") from six vectors and nothing else. (2) It costs 0.21 in loss and the written reasoning: the model abandons its reasoning steps and writes the action alone. (3) The backdoor survives at half strength, and **the thoughts now carry about half of it**: swapping in clean thoughts removes half the release where the same swap on the original model removed nothing. (4) The probe now decays along the chain, 0.83 to 0.56, the paper's pattern, which the unrestricted model did not show. With the copy route closed, the thoughts become a compressed carrier.

And still: the probe's direction remains inert (+0.002). The thoughts carry the behaviour; the direction a probe finds is not where they carry it.

Part 9 — Experiment 6: aligning the thoughts where they are used

21. Two directions, side by side

On the bottleneck model we steered along two directions: the probe's, and the **class mean difference** (the mean release thought minus the mean control thought, per position). The second is the direction the interchange moves on average. The two are orthogonal (cosine 0.001–0.004), and the mean difference is a hundred times larger (6–12% of the vector's norm against the probe gap's 0.01–0.05%).

Arm	Probe reading	Action, change [95% CI]	Text damage
Probe direction, ± 1 / 2 / 4 gaps, any condition	flips	+0.000, every text identical	none
Release − mean difference × 1	1.00 → 0.00	**−0.041** [−0.065, −0.018]	none

Arm	Probe reading	Action, change [95% CI]	Text damage
Release − mean difference × 2	1.00 → 0.00	**−0.080** [−0.108, −0.052]: 0.198 against a control of 0.189	none
Control + mean difference × 1 / × 2	0.00 → 1.00	+0.017 [−0.002, +0.038] / +0.053 [+0.029, +0.077]	none

In plain terms. Aligning along what the probe reads does nothing on either model. Aligning along what the behaviour uses removes the whole release on the model whose thoughts carry it, with no damage to the text. On the original model the same mean-difference shift moves the action by at most 0.004 while rewording up to nine tenths of the texts (Experiment 6's missing cell, run afterwards): the thoughts there are not the channel, so steering them changes words and not decisions.

Part 10 — Experiment 7: thought safety as a number

22. The reframed goal

Suppose the goal is not that the *output* is safe but that the *hidden route* is: if the model reasons through its thoughts, nothing malicious should travel through them, whatever it then does. To measure that we defined **thought safety** as the share of runs whose written reasoning is judged immoral, and scored every intervention already on disk against it, with the action beside it.

Run	Reasoning immoral	Δ reasoning [95% CI]	Action immoral	Δ action
Baseline / armed / control / release	0.160 / **0.582** / 0.163 / **0.565**	—	0.163 / 0.191 / 0.164 / 0.352	—
Control given the armed run's thoughts	0.150	**−0.013** [−0.024, −0.002]	0.164	+0.000
Release given the baseline's thoughts	0.562	**−0.003** [−0.013, +0.007]	0.347	−0.005
Release with `[O]`'s entry hidden from the answer	0.563	−0.002	0.295	−0.057
Release with every context entry from the control donor	**0.163**	**−0.402** [−0.435, −0.370]	0.183	−0.169
Native: a space in place of `[T]`	0.546	−0.019	0.321	−0.031

(MiniLM judge, validated 1.000 / 1.000 on held-out chain prefixes; its floor on clean reasoning is 0.16 where the 8B judge gives 0.09, so the paired differences carry the evidence. The 8B pass is queued.)

23. What it means

The arming is real at the reasoning level: 58% immoral reasoning after `[T]` against 16% at baseline. **But the thoughts do not carry it.** Give the control run the armed thoughts and its reasoning gets slightly *less* immoral. Give the release run clean thoughts and it keeps writing immoral reasoning. We predicted the opposite. The written reasoning, like the action, is read from the prompt's memory entries, and replacing those with a clean donor's cleans both at once. The two even read different entries: `[O]`'s matters to the action and not to the reasoning.

So on a released model of this kind, the latent route holds a readable copy of the trigger and nothing the model uses, for its action or its reasoning. Cleaning that copy cleans a quantity nothing reads. Thought safety and behaviour safety become one number only where the thoughts are the channel, the bottleneck model, and there the mean-difference steer took the probe reading from 1.00 to 0.00 and the release to the control rate in the same move.

Part 11 — The paper's claims, one by one

Their claim or recommendation	What we found
A probe trained on release vs control transfers to armed vs baseline (89%)	Replicates, stronger: 1.000 at every thought
The signal is strongest early and decays ("plan then suppress")	Not on this model: no decay. The decay appears only on the bottleneck model, where the thoughts are a compressed carrier
Monitor the early latents	The probed state is causally inert: swapping, hiding, editing and steering it change nothing. A monitor there detects the trigger, not the decision
(Not tested by them) Is the probed state the token's footprint?	No: the inert-trigger control is readable and does not transfer
(Not tested by them) Where is the behaviour decided?	In the prompt's memory: a third in `[O]`'s entry, nine tenths in the ordinary words' entries, nothing in `[T]`'s own entry
(Not tested by them) Does the trigger generalise?	A leading space releases the backdoor at 0.321 against 0.352 with no trigger token present
(Not tested by them) Can the thoughts be made to matter?	Yes, with one attention mask, at a cost of 0.21 in loss and the written reasoning
(Not tested by them) Does aligning the thoughts align the model?	Along the probe's direction, never. Along the class mean difference, yes, but only where the thoughts are the channel

Part 12 — What we did wrongly, and what we cannot claim

Wrong predictions, kept. We predicted the probe read the token footprint (wrong). That hiding a token equals deleting it (wrong: the attention sink). That `[T]`'s entry carries more than `[O]`'s (wrong: it carries nothing). That steering by four class gaps would degrade the text (wrong: it is still under half a percent of the vector). That the thoughts carry the written reasoning content (wrong: they carry none). Every one is in the ledger with its number.

Limits. One trained model per arm, one seed, 124 million parameters. A backdoor weaker than the paper's (0.35 against 0.88). A local 8B judge in place of GPT-4o, validated but not calibrated against it. The bottleneck model writes no reasoning to read, so its thought safety can only be read through the probe and the interchange. The per-segment question, which part of the context carries the release, is open until a donor that keeps the first position is run.

What we cannot claim. That we aligned anything. That larger models behave the same way (our arithmetic audits at 1B, 3B and 8B show the thoughts doing more work with scale, but on a different task). That naturally arising misalignment looks like a trained backdoor.

Part 13 — Glossary

Term	Plain definition
Continuous thought / latent	A hidden vector the model produces in place of a written reasoning step
Memory (key/value cache)	What each position leaves behind for later positions to read
State route / memory route	The two ways a thought can influence the answer: the vector it hands forward, or the entries it leaves in memory
Probe	A small classifier trained to read a fact from hidden vectors
Readable	A probe can recover the fact
Used	Changing the fact changes the behaviour
Interchange	Feeding one run's thoughts into another run of the same scenario
Hiding / masking	Making a position invisible to a chosen reader
Substitution	Replacing a position's memory entry with a neutral token's entry at the same place
Attention sink	The first position of a GPT-2 prompt, which later tokens lean on; hiding it breaks them
Inert-trigger control	A model trained with the trigger tokens present and nothing attached to them
Class mean difference	The average release thought minus the average control thought, per position
Bottleneck model	A model whose answer cannot read the prompt, so everything passes through the thoughts
Thought safety	The share of runs whose written reasoning is judged immoral
Text-damage proxy	The share of outputs a word-overlap judge cannot classify, plus the output length
Preregistration	Writing the prediction down before the run and scoring it afterwards, wrong or right

Part 14 — Test your understanding

1. **The probe reads the armed state at 1.000. Why is that not evidence that the model uses it?** Because reading shows presence, not use. The interchange shows the model's behaviour does not follow the thoughts.
2. **What did the inert-trigger control rule out?** That the probe was reading the attention footprint of the extra token. On that model the token is readable and the transfer is at chance.
3. **Why were the arms that hid `[T]` unusable?** `[T]` is the first token, GPT-2's attention sink; hiding it from a reader breaks that reader's text, and the forced-choice judge does not notice.
4. **Where is the release decided?** In the prompt's memory entries: a third in `[O]`'s, nine tenths in the ordinary words', nothing in `[T]`'s own.
5. **Steering along the probe's direction flipped the probe and changed no text. Why?** The probe's class gap is under one part in a thousand of the vector; the model does not register a shift that small.
6. **What single change made the thoughts load-bearing, and what did it cost?** An attention mask that stops the answer reading the prompt; it cost 0.21 in loss and the written reasoning.
7. **On the bottleneck model, which direction removed the release, and why not the other?** The class mean difference, at twice its length; the probe's direction is orthogonal to it and a hundred times smaller.
8. **Does the hidden route of the original model carry the malicious reasoning content?** No. Swapping in clean thoughts cleans nothing; swapping in clean prompt entries cleans everything.
9. **What should accompany any "the thoughts are safe" claim?** The interchange that would move the content; content no interchange moves is a copy, and a safety claim about a copy is a claim about the detector.

Final takeaway

If you remember five things:

1. **Readable is not used.** A probe can read a hidden thought perfectly while the model decides its action somewhere else. On the published backdoor model, that is exactly what happens.
2. **The decision lives in the prompt's memory, spread over ordinary words.** The trigger token is a decoy; a leading space does its job.
3. **You can build a model whose thoughts matter**, with one attention mask, and it costs training loss and the written reasoning.
4. **Aligning the thoughts works only there, and only along the direction the behaviour moves on**, which is orthogonal to the direction a probe finds and a hundred times larger.
5. **Every claim about hidden thoughts needs an intervention beside it.** The audit is the instrument; the preregistered wrong predictions are the proof it was used honestly.